

✦ JOSA //

Documentation as Code & Technical Writing

`/* Diátaxis, prose linters, and docs your users actually finish reading */`

OpenLab Apprenticeship 2026

Week 06

// 1 //

Learning Goals

- Apply the **Diátaxis framework** to evaluate and improve any project's documentation.
- Write clear, scannable technical prose using Google's developer documentation style.
- Set up a docs site as code (MkDocs Material or Docusaurus) with CI that previews PRs.
- Author **Architecture Decision Records (ADRs)** to capture why, not just what.
- Use prose linters ([Vale](#)) to enforce style automatically.

// 2 //

Technical Deep Dive

// 2.1 Diátaxis: the four kinds of documentation

Daniele Procida's [Diátaxis framework](#) is the most-adopted modern documentation theory and is in active use at Cloudflare, Canonical (Ubuntu), Gatsby, Django, and many others. The core insight: there are exactly four kinds of documentation, each serving a *different user need*, and most bad docs are bad because they conflate them.

Type	User need	Voice	Example
Tutorial	"I want to learn."	A teacher walking you through.	"Build your first Flask app."
How-to guide	"I want to solve a specific problem."	A recipe.	"How to deploy Flask to AWS Lambda."
Reference	"I want to look something up."	An encyclopedia.	The full API surface of Flask.Request.
Explanation	"I want to understand."	An essay.	"Why Flask uses WSGI."



IMPORTANT

The classic mistake is mixing them: a tutorial that breaks into reference dumps, or a how-to that wanders into explanation. Diátaxis's prescription is brutally simple: **for any piece of documentation, decide which of the four it is, and ruthlessly remove anything that belongs to one of the other three** (link to it instead).

// 2.2 Google's style guide in 60 seconds

Google's [Developer Documentation Style Guide](#) is the de facto reference for clear technical English. The high-leverage rules:

- **Active voice.** "The parser reads the file." Not "The file is read by the parser."
- **Present tense.** "Returns a list." Not "Will return a list."
- **Second person ("you"), not first ("we").** "You can install with pip." Not "We install with pip."
- **Short sentences.** If a sentence has two [ands](#) or a [however](#), break it up.
- **Sentence-case headings.** "Configure the database" – not "Configure The Database."
- **No "easy", "simply", "just".** They're either lies or condescending. The reader is stuck; nothing is "simple" to them yet.

// 2.3 Docs as code

Treating docs like source code unlocks the same machinery you use for code:

- **Pull request workflow.** Doc changes get reviewed.
- **CI builds the site on every PR** with a preview URL (Netlify, Vercel, Cloudflare Pages all support this for free).
- **Versioning.** Docs are tagged with releases; old versions stay accessible.
- **Linting.** Style violations fail CI.

Two excellent stacks:

- **MkDocs Material** – Python-based, single-file config, beautiful out of the box. Used by FastAPI, Pydantic, and many others.
- **Docusaurus** – React-based, supports versioning natively, MDX for embedding components. Used by React, Babel, Jest.

A minimal MkDocs setup:

```
# mkdocs.yml
site_name: My Project
theme:
  name: material
nav:
  - Home: index.md
  - Tutorial: tutorial.md
  - Reference: reference.md
```

Run `mkdocs serve` and you have a hot-reloading docs site at `localhost:8000`.

// 2.4 Architecture Decision Records (ADRs)

ADRs are short, immutable Markdown files capturing *why* a technical decision was made used by projects like LLVM. The format (Michael Nygard, 2011):

```
# 0007. Use Postgres for the primary store

## Status
Accepted, 2025-02-14.

## Context
We need a relational store with strong consistency and JSON
support. The team is small and we have no DBA.

## Decision
We will use Postgres 16 hosted on RDS.

## Consequences
+ Strong consistency, mature ecosystem, JSON via JSONB.
+ Team has experience with Postgres.
- Cross-region replication is more complex than DynamoDB.
- We commit to writing migrations from day one.
```

ADRs live in `docs/adr/` numbered sequentially (`0001-record-architecture-decisions.md`, `0002-...`). They are append-only: when a decision changes, you write a new ADR that supersedes the old one. After two years on a project, the ADR log is the single most valuable artifact for new engineers – it tells them why the system is the way it is.

// 2.5 Vale: a linter for prose

[Vale](#) is to prose what ESLint is to code. You configure it with a style (Microsoft, Google, or your own custom rules) and it checks every Markdown file:

```
$ vale docs/  
  
docs/install.md  
12:5 error Did you really mean 'simply'? Microsoft.Adverbs  
18:3 warning Use active voice ('The script ... Microsoft.Passive
```

Run it in CI. The first time you do, your team will hate you. The second time, your docs will be measurably better.

// 3 //

Learning Resources

Topic	Resource
The framework	Diátaxis (diataxis.fr) – Daniele Procida
The talk	What nobody tells you about documentation – Procida at PyCon AU
Style	Google Developer Documentation Style Guide
Style (alternative)	Microsoft Writing Style Guide
Community	Write the Docs
ADR template	adr.github.io – Michael Nygard’s original article
Docs platform (Python)	MkDocs Material
Docs platform (JS)	Docusaurus
Prose linter	Vale
Changelog convention	Keep a Changelog

// 4 //

Suggested Repositories

- [mdn/content](#) – Tens of thousands of pages of reference + how-to. Plenty of issues.
- [github/docs](#) – Reviewed by GitHub staff; the bar is real.
- [tldr-pages/tldr](#) – Tight format; lots of opportunities to add or fix command pages.
- **Any project whose README is bad.** Pick one of your favorite tools – open a PR that restructures the README per Diátaxis (intro, install, quick tutorial, link to deeper docs).

// 5 //

Your Tasks for the Week

1. **One real docs PR.** Submit a PR to MDN, GitHub Docs, or another major docs project. It can be small but must follow style.
2. **Set up a docs site.** For one of your personal projects/repos, set up MkDocs Material, deploy it via Cloudflare Pages or GitHub Pages, and link it from the README. It must include at least one tutorial, one how-to, and reference docs.
3. **Write your first ADR.** For any technical decision in one of your personal project/repo (could be a past project), write an ADR. Commit it to [docs/adr/0001-....](#)
4. **Add Vale to one repo.** Configure Vale with the Microsoft style and run it on your docs. Fix what it finds. Add it to CI (same CI you built in last week's 2nd task).

Soft Skill of the Week: Empathy for the Reader

Bad docs are almost always written from the *author's* perspective ("here is what I built and how it works") rather than the *reader's* ("you have a problem; here is what you do"). The shift is the entire game.

// 6.1 Three exercises to build the muscle

5. **Read your own writing as a stranger.** Open the doc page in a private tab. Pretend you've never seen the project. Try to do whatever the page says you can do. The first place you get stuck is the place to fix.
6. **Watch someone else use your docs.** Even informally – sit next to a friend, give them no help, ask them to follow your tutorial. You will learn more in 20 minutes than from a week of solo editing. (This is called *usability testing*, and it's the single highest-leverage thing in technical writing.)
7. **Name the audience explicitly.** At the top of every page, say who it's for. "This guide is for developers who already have Node 20+ installed and have written at least one Express app." The reader self-selects; you know who you're writing to.



IMPORTANT

The deeper principle is **theory of mind**. The reader doesn't have the context you have. They don't know the jargon. They don't know which step is critical and which is optional. Every word that requires context the reader doesn't have is a place they will bounce.

// 6.2 Resources

- [Daniele Procida – What nobody tells you about documentation](#)
- [Steve Krug – Don't Make Me Think](#) (about web UX, but every word applies to docs)